JOBSHEET 13

MIDDLEWARE & ROUTE PROTECTION

PEMROGRAMAN BERBASIS FRAMEWORK

REIKA AMALIA SYAHPUTRI

2341720173

TI-3E / 19

TEKNOLOGI INFORMASI

Github:

https://github.com/ReikaAmalia/Pemrograman_Berbasis_Framework/tree/main/praktikum_13

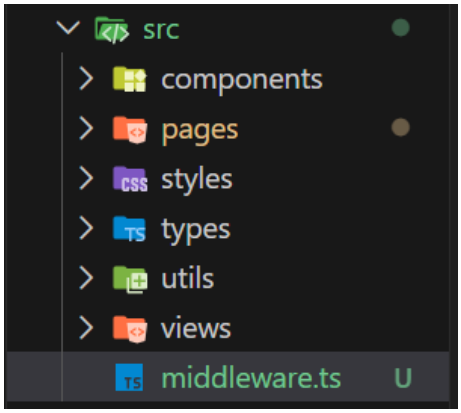
C. Langkah Praktikum

Bagian 1 – Membuat Middleware

- Modifikasi file index.tsx pada folder src/pages/produk (index.tsx)

```
import { useRouter } from "next/router";
import { useEffect, useState } from "react";
import TampilanProduk from "@/views/produk";
import useSWR from "swr";
import fetcher from "../../utils/swr/fetcher";

const kategori = () => {
    const { push } = useRouter();
    const [products, setProducts] = useState([]);


    const { data, error, isLoading } = useSWR("/api/produk", fetcher);
    // cek apakah data, error, dan isLoading sudah bnar

    return (
        <div>
            <TampilanProduk
            products={isLoading ? [] : data.data} />
        </div>
    );
};

export default kategori;
```

- Buat file: src/middleware.ts Sejajar dengan folder pages.



Bagian 2 – Struktur Dasar Middleware

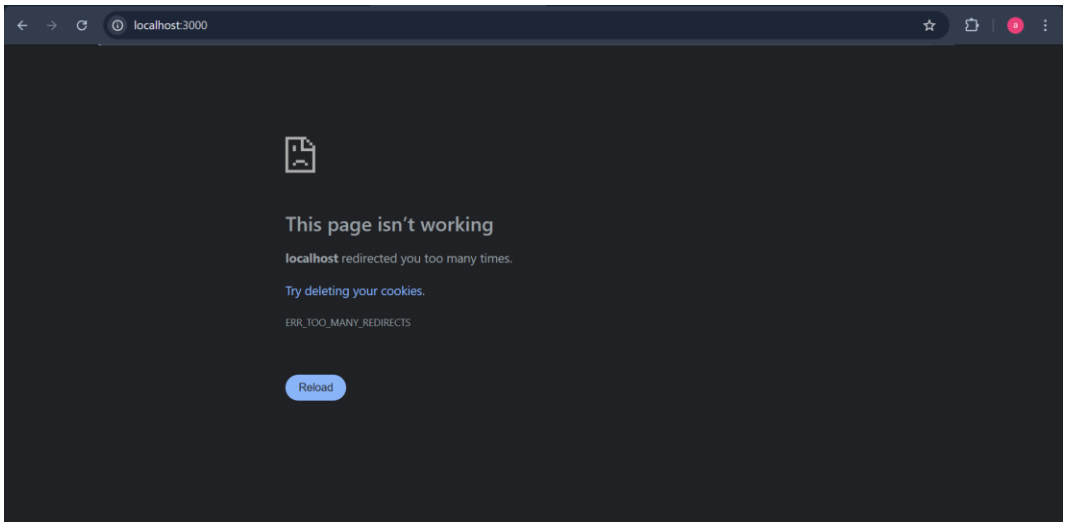
```
middleware.ts M ×
my-app-13 > src > middleware.ts > middleware
1  import { NextResponse } from "next/server";
2  import type { NextRequest } from "next/server";
3
   Windsurf: Refactor | Explain | Generate JSDoc | ×
4  export function middleware(request: NextRequest) {
5      return NextResponse.next();
6  }
```

- Jika menggunakan NextResponse.next() → tidak ada redirect.
- Jadi masih bisa mengakses ke http://localhost:3000/produk

Bagian 3 – Redirect Sederhana

```
1  import { NextResponse } from "next/server";
2  import type { NextRequest } from "next/server";
3
4  export function middleware(request: NextRequest) {
5      return NextResponse.redirect(new URL("/", request.url));
6
7      //return NextResponse.next();
8  }
```

- Semua halaman akan redirect ke home dan error dikarenakan terus menerus loading



Bagian 4 – Batasi Route Tertentu

- Untuk mengatasi pada bagian 3 maka perlu pembatasan route

```
import { NextResponse } from "next/server";
import type { NextRequest } from "next/server";

export function middleware(request: NextRequest) {
    return NextResponse.redirect(new URL("/", request.url));

    //return NextResponse.next();
}

    export const config = {
        matcher: "/about/:path*",
    };

```

Artinya:

- Middleware hanya berlaku untuk /products dan /about
- Halaman lain tetap normal
- Ketika user mengakses halaman produk dan about maka akan langsung redirect ke halaman home

Bagian 5 – Simulasi Sistem Login

- Modifikasi file middleware.ts

```
import { NextResponse } from "next/server";
import type { NextRequest } from "next/server";

export function middleware(request: NextRequest) {
  const isLogin = false;
  if (isLogin) {
    return NextResponse.next();
  } else {
    return NextResponse.redirect(new URL("/auth/login", request.url));
  }

  //return NextResponse.redirect(new URL("/", request.url));
  //return NextResponse.next();
}

export const config = {
  matcher: ["/produk", "/about"],
};

```

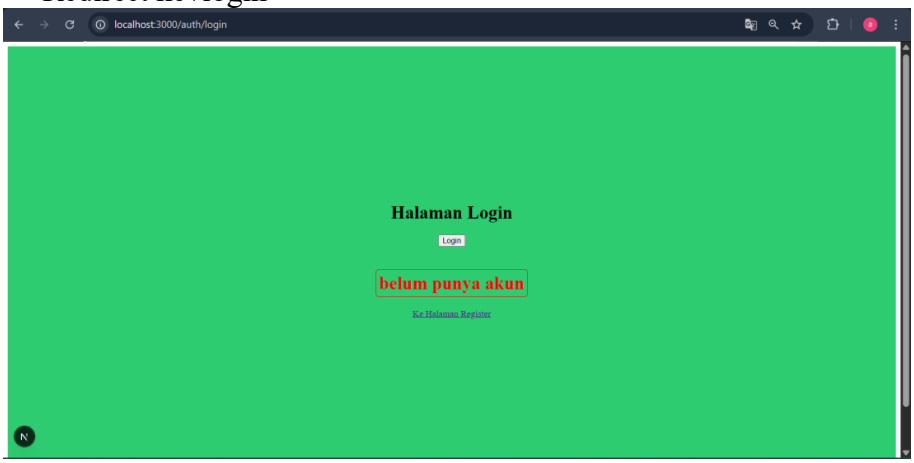
- Jika user langsung mengakses ke alamat http://localhost:3000/produk tidak akan bisa user akan diarahkan ke halaman login

B. Pengujian

- Uji 1 – isLogin = false

```
import { NextResponse } from "next/server";
import type { NextRequest } from "next/server";

export function middleware(request: NextRequest) {
  const isLogin = false;
  if (isLogin) {
    return NextResponse.next();
  } else {
    return NextResponse.redirect(new URL("/auth/login", request.url));
  }

  //return NextResponse.redirect(new URL("/", request.url));
  //return NextResponse.next();
}

export const config = {
  matcher: ["/produk", "/about"],
};

```

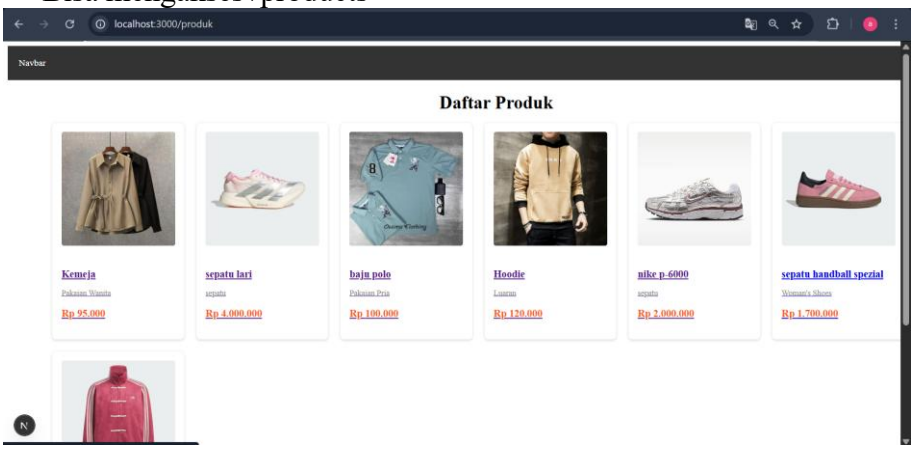
- Akses:
  /products
- Hasil:
  ➜ Redirect ke /login



- Uji 2 – isLogin = true
- Ubah:
  const isLogin = true

```
middleware.ts M ×
my-app-13 > src > middleware.ts > middleware > isLogin
1   import { NextResponse } from "next/server";
2   import type { NextRequest } from "next/server";
3
    Windsurf: Refactor | Explain | Generate JSDoc | ×
4   export function middleware(request: NextRequest) {
5     const isLogin = true;
6     if (isLogin) {
7       return NextResponse.next();
8     } else {
9       return NextResponse.redirect(new URL("/auth/login", requ
10    }
11
12    //return NextResponse.redirect(new URL("/", request.url));
13    //return NextResponse.next();
14  }
15
16  export const config = {
17    matcher: ["/produk", "/about"],
18  };
```

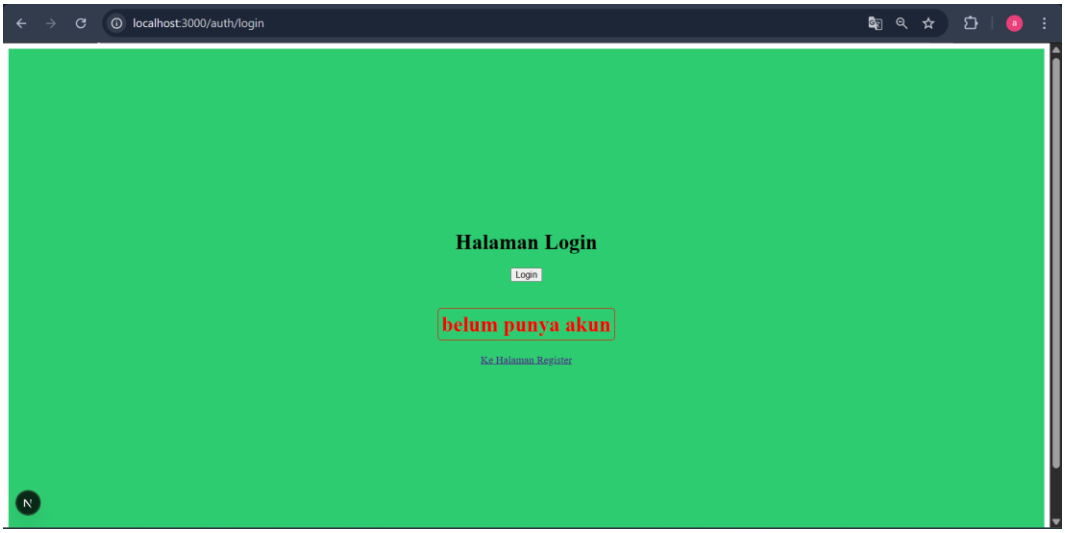
- Hasil:
  ➜ Bisa mengakses /products



- Uji 3 – Tambahkan Multiple Route export
  const config = {
  matcher: ['/products', '/about']
  }

```python
import { NextResponse } from "next/server";
import type { NextRequest } from "next/server";

export function middleware(request: NextRequest) {
  const isLogin = false;
  if (isLogin) {
    return NextResponse.next();
  } else {
    return NextResponse.redirect(new URL("/auth/login", request.url));
  }

  //return NextResponse.redirect(new URL("/", request.url));
  //return NextResponse.next();
}

export const config = {
  matcher: ["/produk", "/about"],
};

```

- Sekarang:
  /products dan /about butuh login
  Halaman lain bebas



C. Perbandingan Middleware vs useEffect

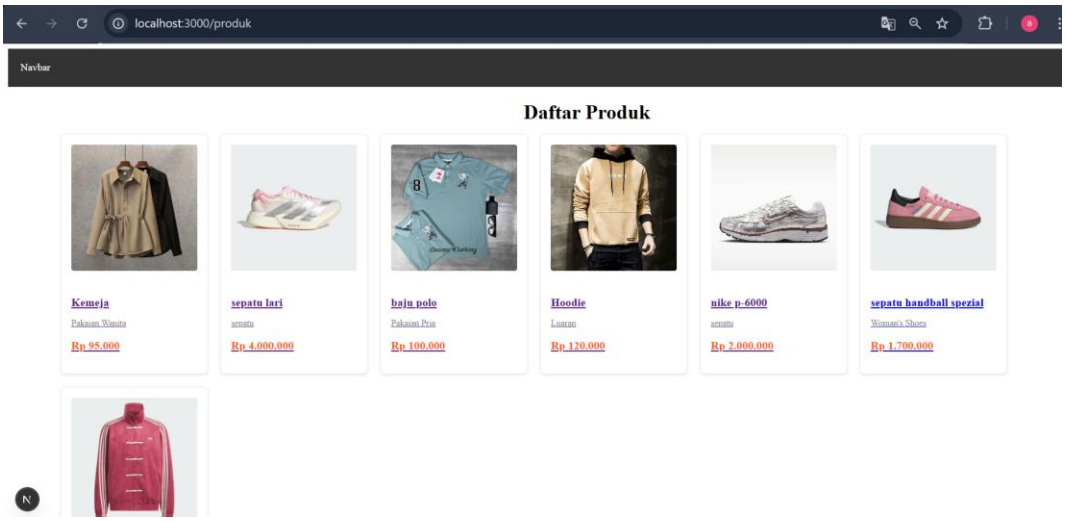
Aspek	useEffect	Middleware
Redirect timing	Setelah render	Sebelum render
Glitch	Ada	Tidak
Security	Lemah	Lebih aman
Skalabilitas	Harus tiap halaman	Sekali di middleware

D. Tugas Praktikum

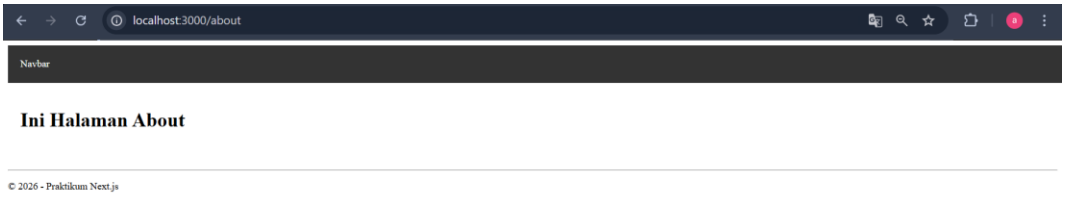
- Tugas Individu

1. Buat halaman:

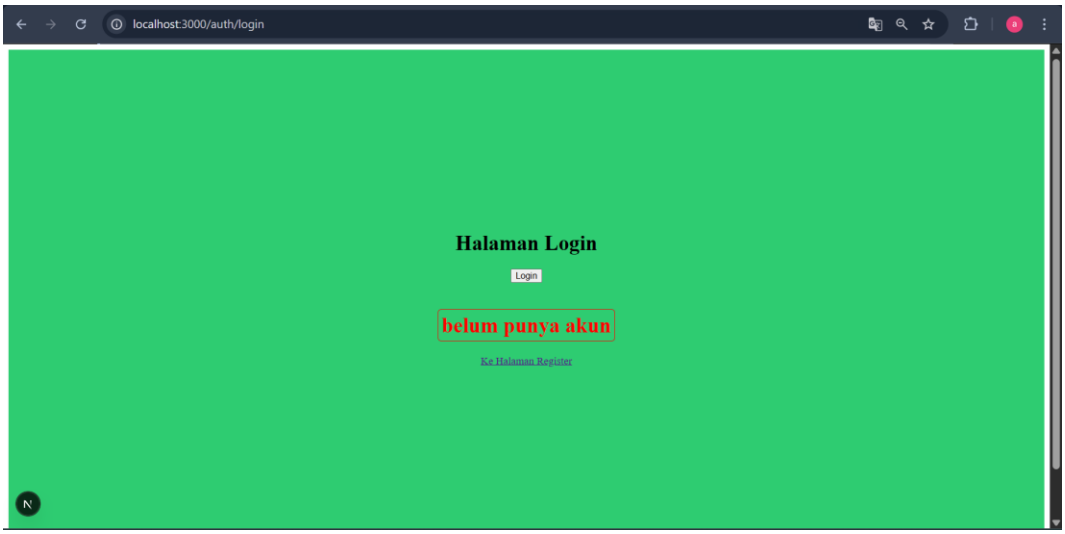
   /produk



   /about



/login



2. Implementasikan Middleware:

- Redirect ke /login jika belum login.
- Izinkan akses jika login true.

```
import { NextResponse } from "next/server";
import type { NextRequest } from "next/server";

export function middleware(request: NextRequest) {

  const isLogin = request.cookies.get("isLogin")?.value;

  if (isLogin === "true") {
    return NextResponse.next();
  } else {
    return NextResponse.redirect(new URL("/auth/login", request.url));
  }
}

export const config = {
  matcher: ["/produk", "/about"],
};
```

3. Tambahkan proteksi hanya untuk route tertentu.

   Proteksi hanya diterapkan pada route tertentu dengan menggunakan matcher pada middleware. Halaman seperti /produk dan /about memerlukan login, sedangkan halaman lain seperti halaman login tetap dapat diakses tanpa autentikasi. Hal ini bertujuan agar tidak semua halaman dibatasi, sehingga user tetap dapat mengakses halaman umum tanpa login.


```
import { NextResponse } from "next/server";
import type { NextRequest } from "next/server";

export function middleware(request: NextRequest) {

  const isLogin = request.cookies.get("isLogin")?.value;

  if (isLogin === "true") {
    return NextResponse.next();
  } else {
    return NextResponse.redirect(new URL("/auth/login", request.url));
  }
}

export const config = {
  matcher: ["/produk", "/about"],
};
```


4. Dokumentasikan:

- Screenshot sebelum dan sesudah redirect.
- Perbandingan dengan useEffect.

G. Pertanyaan Analisis

1. Mengapa middleware lebih aman dibanding useEffect?
   - Middleware lebih aman karena proses pengecekan dilakukan sebelum halaman ditampilkan ke user. Artinya, user yang belum memiliki akses tidak akan pernah melihat isi halaman sama sekali. Sedangkan useEffect berjalan di sisi client setelah halaman dirender, sehingga halaman sempat terbuka walaupun akhirnya di-redirect.
2. Mengapa middleware tidak menimbulkan glitch?
   - Middleware tidak menimbulkan glitch karena redirect dilakukan sebelum proses render halaman. Jadi halaman yang tidak memiliki izin tidak pernah tampil. Berbeda dengan useEffect, di mana halaman sempat muncul sebentar sebelum dialihkan, sehingga terlihat seperti berkedip atau glitch.
3. Apa risiko jika semua halaman diproteksi tanpa pengecualian?
   - Jika semua halaman diproteksi, maka user tidak bisa mengakses halaman penting seperti login atau register. Hal ini dapat menyebabkan redirect terus-menerus (infinite redirect) dan membuat aplikasi tidak bisa digunakan dengan normal.
4. Kapan middleware tidak diperlukan?
   - Middleware tidak diperlukan ketika halaman tidak membutuhkan proteksi, misalnya halaman publik seperti home, about umum, atau landing page. Selain itu, untuk aplikasi sederhana yang hanya membutuhkan pengecekan ringan di client, useEffect masih bisa digunakan.
5. Apa perbedaan middleware dan API route?
   - Middleware digunakan untuk memproses request sebelum halaman ditampilkan, biasanya untuk keperluan seperti autentikasi dan redirect. Sedangkan API route digunakan untuk membuat endpoint backend, misalnya untuk mengambil data, menyimpan data, atau berinteraksi dengan database.